

SENIOR

Target Date: 08/08/2026

Senior Full Stack Engineer responsible for building web applications using React, Next.js App Router, Node.js, and PostgreSQL, leading microservices system design, and mentoring junior engineers.

1. QUICK REVISION SHEET

Key Focus Takeaways:

- State time & space complexity before coding.
- Highlight production resilience and trade-offs.

Spoken Answer Strategy:

- Structure answers: Core Answer ' Trade-off ' Result.
- Keep verbal model responses between 90 & 150 seconds.

2. TERMS TO REMEMBER

System Design

Scalability

Concurrency

Database Indexing

API Security

3. QUESTIONS & MODEL ANSWERS (5)

Q1

Technical

In Next.js (App Router), how do React Server Components (RSC) change your approach to data fetching, security, and state management compared to traditional Client Components? Give a specific scenario where you would explicitly use a Client Component over a Server Component.

(PRECISE VERBAL MODEL ANSWER:

"React Server Components shift rendering, data fetching, and sensitive execution to the server by default, drastically minimizing client bundle sizes and securing data boundaries."

"RSCs shift execution to the server by default. For data fetching, direct async/await inside components eliminates client JS overhead and network waterfalls. Security improves because API keys and DB calls remain strictly server-side. For state management, we replace bloated client stores with URL parameters and Server Actions paired with tag revalidation. I explicitly opt for a Client Component using 'use client' at the leaf level whenever I need browser interactivity—such as an interactive search bar requiring local useState, useEffect, and event listeners for debouncing."

Q2

System Design

Walk me through how you would architect a scalable microservices backend using Node.js, GraphQL, and PostgreSQL. How do you handle schema federation, database migration management across services, and inter-service communication?

(PRECISE VERBAL MODEL ANSWER:

"I would architect a domain-driven microservices ecosystem using Apollo Federation for unified GraphQL querying, isolated PostgreSQL databases per service, and a hybrid async/sync communication backbone."

"I'd approach this using a domain-driven microservices architecture where individual Node.js services act as independent subgraphs. At the edge, an Apollo Router composes these subgraphs into a unified GraphQL supergraph, giving clients a single endpoint while letting teams deploy schemas autonomously. For data persistence, I strictly enforce a database-per-service pattern using PostgreSQL to eliminate database coupling. Migration management is fully automated within CI/CD pipelines using the Expand-Contract pattern—like adding new columns as nullable first—to guarantee zero-downtime deployments. For inter-service communication, I use high-performance gRPC for rare synchronous calls, but rely primarily on asynchronous event streaming via Kafka for state changes. To avoid distributed transaction failures, I implement the Transaction Outbox pattern in PostgreSQL to reliably publish domain events. The primary trade-off is operational complexity in federated schema checks and tracing, which I mitigate using automated CI/CD schema validation and OpenTelemetry."

Q3

Problem Solving

Suppose your Node.js backend is experiencing high latency and connection pool starvation when querying a PostgreSQL database under heavy traffic. How would you diagnose, isolate, and remediate this bottleneck?

ANSWER OUTLINE:

Describe using EXPLAIN ANALYZE to identify unindexed scans, checking for long-running transactions, introducing connection pooling via PgBouncer or adjusting pool sizes in Node, optimizing query execution, and using caching strategies (e.g., Redis).

Q4

Behavioural

As a Senior Engineer leading code reviews and mentorship, tell me about a time you identified significant architectural flaws in a junior engineer's pull request. How did you deliver actionable feedback without demotivating them?

ANSWER OUTLINE:

Structure answer using STAR method. Describe the architectural issue, emphasize framing feedback constructively (focusing on code performance/maintainability rather than personal critique), pairing with the engineer to guide them toward a solution, and the positive outcome on both code quality and individual growth.

Q5

Domain Knowledge

When deploying a high-traffic Next.js and Tailwind CSS application on Vercel or AWS, what strategies do you employ to ensure optimal Core Web Vitals, robust web accessibility (a11y), and zero-downtime CI/CD deployments?

ANSWER OUTLINE:

Cover Core Web Vitals optimization (Next.js Image/Font optimization, dynamic imports, layout stability via Tailwind), semantic HTML and ARIA attributes for accessibility, edge caching/ISR strategies on Vercel/AWS CloudFront, and automated CI/CD checks (Lighthouse CI, automated linting/tests).